PromptOps: AI Prompt Management Platform

Design Document

---

1. Introduction

1.1 Purpose

This design document describes the architecture, components, and implementation plan for **PromptOps**, a web-based SaaS platform for managing, versioning, testing, and deploying AI prompts. This document serves as the complete technical specification and implementation guide for a solo developer building this project.

The document aims to:

- Define the system architecture and component interactions
- Specify database schemas, API endpoints, and data flows
- Establish a realistic implementation timeline
- Document technology choices and design decisions
- Provide guidance for testing, deployment, and maintenance

1.2 Scope

In Scope:

- Multi-tenant web application for prompt management
- Full CRUD operations for prompts with automatic versioning
- Interactive testing sandbox supporting OpenAI and Anthropic APIs
- Basic analytics dashboard (usage metrics, costs)
- User authentication and role-based access control
- RESTful API for all operations
- Responsive web interface
- GitHub integration for prompt backup/sync
- Deployment pipeline with basic automation
- Docker-based deployment

Out of Scope (Future Enhancements):

- Native mobile applications (iOS/Android)
- Real-time collaboration features (live editing)
- Advanced analytics and machine learning insights
- Marketplace for sharing prompts

- SAML/Enterprise SSO integration
- Multi-region deployment
- Advanced pipeline orchestration (complex conditional logic)
- White-label/custom branding options

1.3 Definitions and Acronyms

Term	Definition
Prompt	A text instruction sent to an LLM to generate a response
LLM	Large Language Model (e.g., GPT-4, Claude)
Tenant	An organization/account in the multi-tenant system
Version	A snapshot of a prompt's content at a specific point in time
Test Run	An execution of a prompt against an LLM with recorded results
Pipeline	An automated workflow for testing and deploying prompts
RBAC	Role-Based Access Control
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token (for authentication)
CRUD	Create, Read, Update, Delete operations
SaaS	Software as a Service
MVP	Minimum Viable Product

1.4 Reference Materials

- Django Documentation: https://docs.djangoproject.com/
- Django REST Framework: https://www.django-rest-framework.org/
- OpenAI API Documentation: https://platform.openai.com/docs/api-reference
- Anthropic API Documentation: https://docs.anthropic.com/
- PostgreSQL Documentation: https://www.postgresql.org/docs/
- React Documentation: https://react.dev/
- Multi-tenancy Best Practices: https://docs.aws.amazon.com/whitepapers/latest/saas-architecture-fundamentals/
- Prompt Engineering Guide: https://www.promptingguide.ai/

2. Problem Statement and Goals

2.1 Problem Statement

As AI adoption grows, organizations and individual developers are creating numerous prompts for various use cases. However, they face several challenges:

1. **No Version Control**: Prompts are often stored in code comments, spreadsheets, or text files with no systematic versioning, making it difficult to track changes or revert to working versions.
2. **Inconsistent Testing**: Testing prompts across different LLMs is manual and time-consuming, with no standardized way to compare results or track performance over time.
3. **Poor Collaboration**: Teams lack tools for reviewing, commenting on, and approving prompt changes, leading to uncoordinated modifications and quality issues.
4. **No Visibility**: Organizations cannot track prompt usage, costs, or performance metrics, making optimization difficult.
5. **Deployment Challenges**: Moving prompts from development to production is ad-hoc, without validation or rollback capabilities.
6. **Scattered Management**: Prompts are distributed across multiple projects and repositories with no central library for discovery and reuse.

2.2 Goals

Primary Goals:

1. **Centralized Prompt Repository**: Provide a single source of truth for all organizational prompts with search and categorization.
2. **Automatic Version Control**: Track every change to prompts with diff visualization and one-click rollback capabilities.
3. **Interactive Testing Environment**: Enable users to test prompts against multiple LLMs (OpenAI, Anthropic) and compare results side-by-side.
4. **Usage Analytics**: Track metrics including test runs, token consumption, costs, and response times with visual dashboards.
5. **Team Collaboration**: Support commenting, review workflows, and approval processes for prompt changes.
6. **Simple Deployment**: Provide basic pipeline automation for moving prompts through testing to production.

Success Metrics:

- Support 100+ active users in MVP phase
- Handle 10,000+ prompt test runs per month
- Achieve <200ms API response time (p95)

- Support 3+ LLM providers
- 99% uptime for hosted service

2.3 Non-Goals

The following are explicitly **NOT** goals for the initial version:

1. **Advanced AI Features**: No automatic prompt optimization, suggestion systems, or ML-based improvements.
2. **Real-time Collaboration**: No live multi-user editing or presence indicators (like Google Docs).
3. **Mobile Native Apps**: Web-responsive only; native iOS/Android apps are future work.
4. **Enterprise Features**: No SAML SSO, advanced compliance features, or custom SLAs in MVP.
5. **Marketplace**: No public sharing or monetization of prompts between tenants.
6. **Advanced Analytics**: No predictive analytics, anomaly detection, or complex BI integrations in MVP.
7. **Multi-language Support**: English-only interface initially.

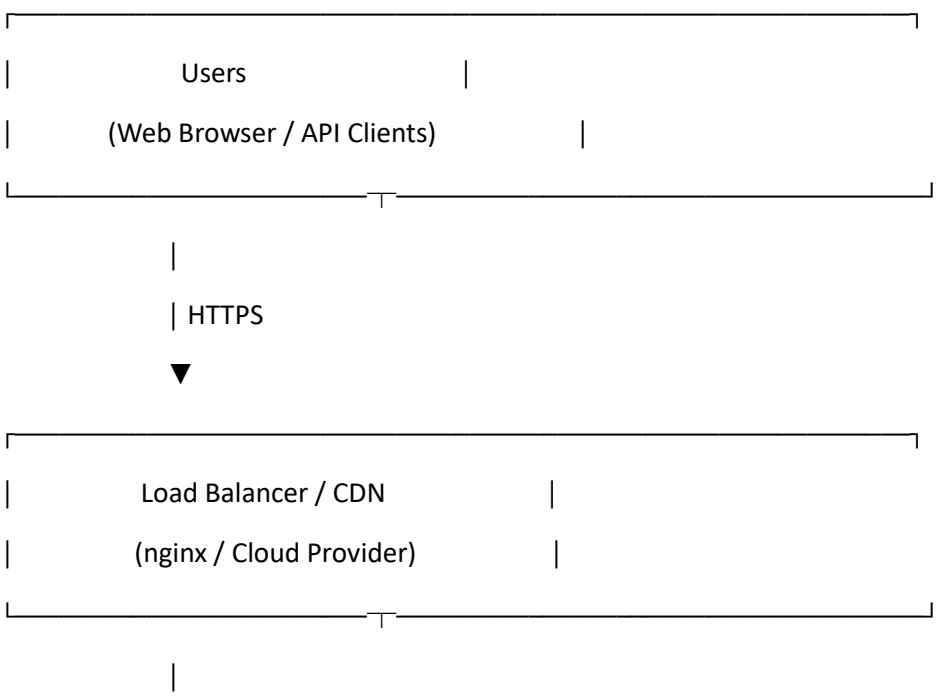
---

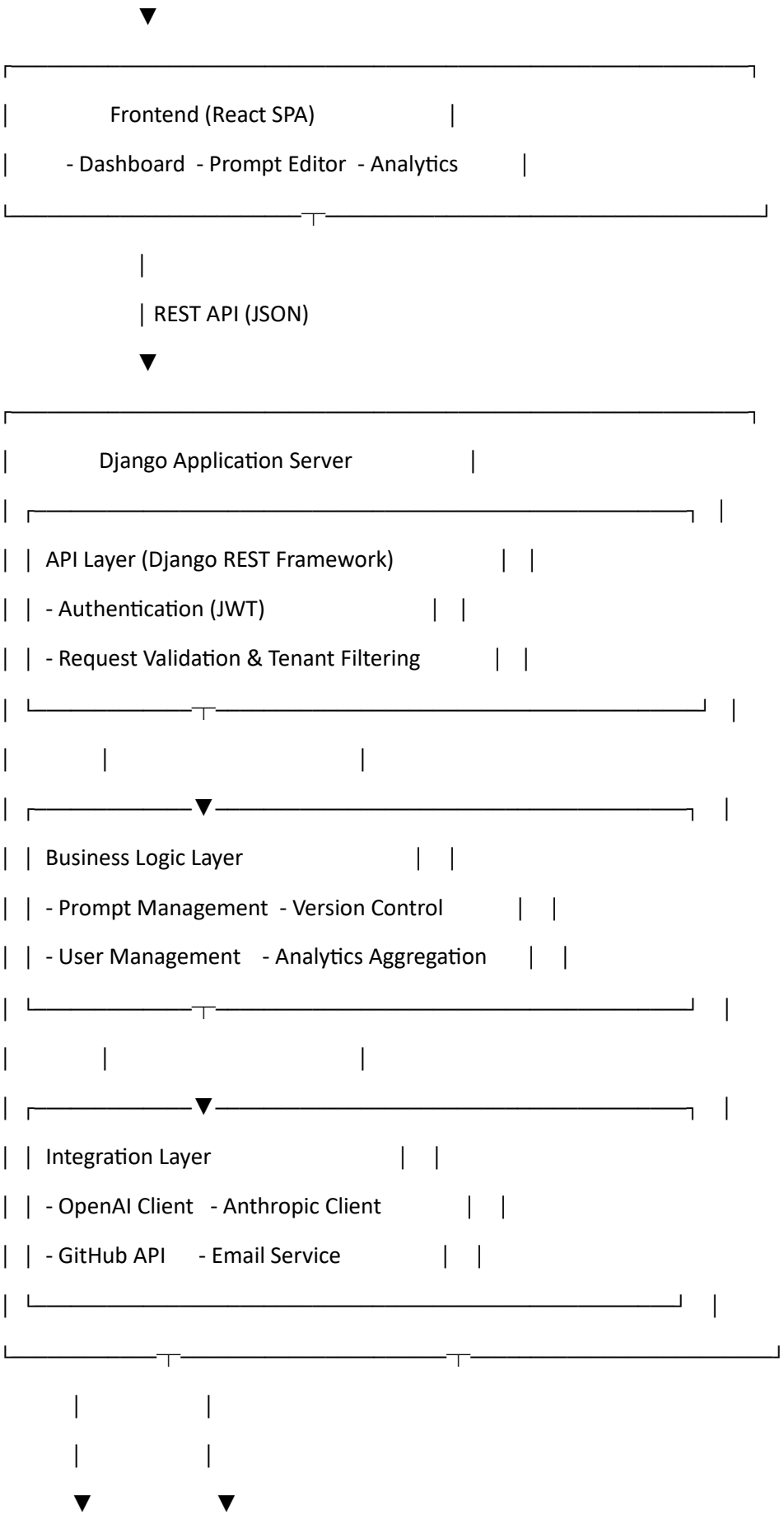
3. System Architecture

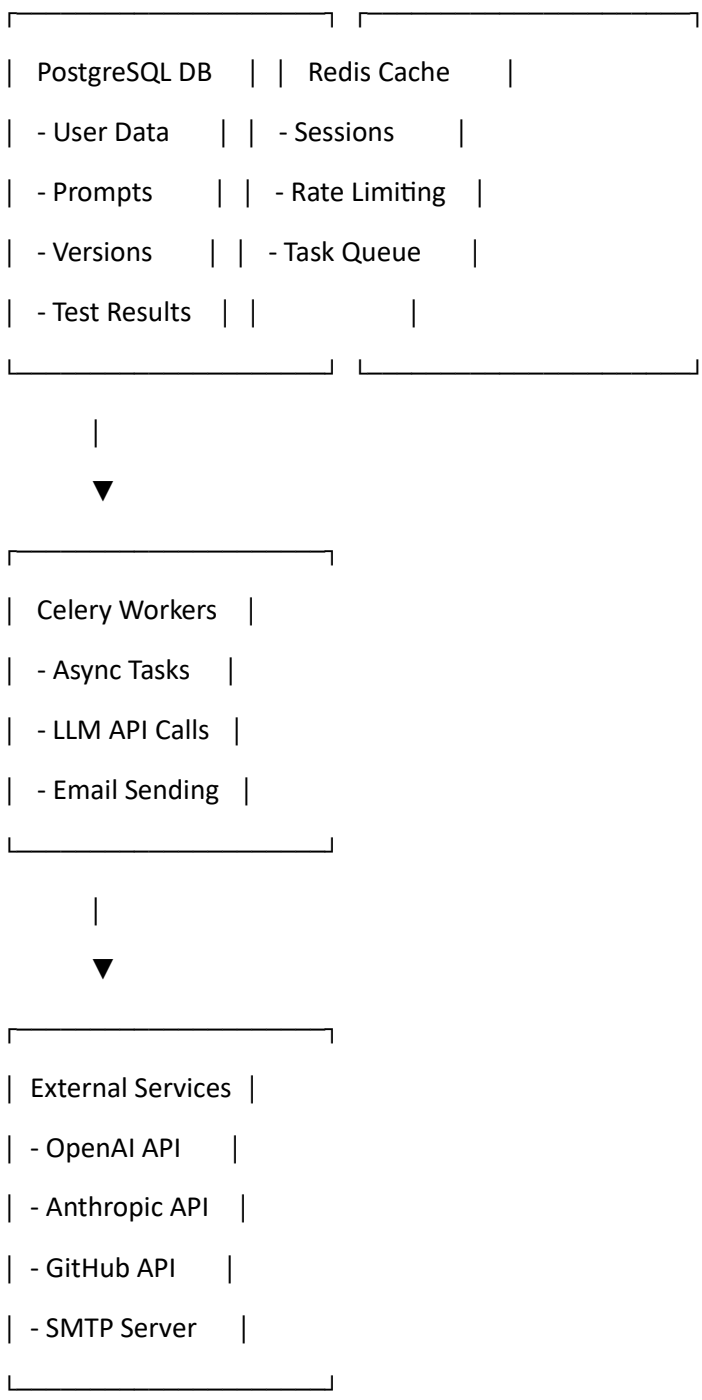
3.1 High-Level Overview

PromptOps follows a **three-tier web architecture** with a Django backend, PostgreSQL database, and React frontend. The system is designed as a **multi-tenant SaaS application** where multiple organizations share the same infrastructure while maintaining complete data isolation.

Architecture Diagram (Text Description):




Frontend (React SPA)
- Dashboard - Prompt Editor - Analytics
REST API (JSON)
Django Application Server
API Layer (Django REST Framework)
- Authentication (JWT)
- Request Validation & Tenant Filtering
Business Logic Layer
- Prompt Management - Version Control
- User Management - Analytics Aggregation
Integration Layer
- OpenAI Client - Anthropic Client
- GitHub API - Email Service



Key Architectural Patterns:

1. **Multi-Tenancy**: Single database with tenant_id column on all tables. Every query filters by tenant to ensure data isolation.
2. **RESTful API**: Stateless HTTP API with JWT authentication. All client-server communication via JSON.
3. **Async Task Processing**: Long-running operations (LLM calls, bulk imports) handled by Celery workers to keep API responsive.

4. **Caching Strategy**: Redis caches user sessions, frequently accessed prompts, and analytics aggregations to reduce database load.
5. **Service Layer Pattern**: Business logic separated from views/serializers for better testability and reusability.

3.2 System Components

3.2.1 Frontend (React SPA)

Purpose: Provides user interface for all PromptOps functionality.

Key Features:

- Dashboard with prompt library and quick stats
- Prompt editor with syntax highlighting
- Version comparison viewer (side-by-side diff)
- Testing sandbox with LLM provider selection
- Analytics charts (usage over time, cost breakdown)
- Team and user management interfaces
- Settings and configuration panels

Technology Stack:

- React 18+ with hooks
- React Router for navigation
- TanStack Query (React Query) for API state management
- Tailwind CSS for styling
- Recharts for data visualization
- Monaco Editor for code editing
- Axios for HTTP requests

Routing Structure:

/ → Dashboard/Home

/prompts → Prompt Library List

/prompts/new → Create New Prompt

/prompts/:id → Prompt Detail View

/prompts/:id/edit → Edit Prompt

/prompts/:id/versions → Version History

/prompts/:id/test → Testing Sandbox

/analytics → Analytics Dashboard

/teams → Team Management

/settings → User/Org Settings

/login → Login Page

/register → Registration

3.2.2 Backend API (Django)

Purpose: Core application server handling business logic, data persistence, and external integrations.

Django Apps Structure:

1. **accounts** - User, tenant, and team management
    - User authentication and registration
    - Tenant provisioning
    - Team CRUD operations
    - Permission checking
2. **prompts** - Core prompt management
    - Prompt CRUD operations
    - Version creation and management
    - Test run execution and recording
    - Comment system
3. **analytics** - Metrics and reporting
    - Usage statistics aggregation
    - Cost calculations
    - Dashboard data preparation
    - Export functionality
4. **integrations** - External service connections
    - LLM provider clients (OpenAI, Anthropic)
    - GitHub API integration
    - Email service
    - Webhook handlers

Middleware Components:

- **TenantMiddleware**: Extracts tenant context from request (subdomain/header)
- **AuditMiddleware**: Logs all significant actions to audit trail
- **RateLimitMiddleware**: Prevents abuse via Redis-based rate limiting

3.2.3 Database (PostgreSQL)

Purpose: Primary data store for all application data.

Key Features:

- ACID compliance for data integrity
- JSON field support for flexible metadata
- Full-text search capabilities
- Robust indexing for query performance
- Connection pooling (via pgBouncer if needed)

Data Organization:

- Single database, single schema
- All tables include tenant_id for isolation
- Indexes on tenant_id + frequently queried fields
- Foreign key constraints for referential integrity

3.2.4 Cache Layer (Redis)

Purpose: High-performance caching and task queue.

Use Cases:

1. **Session Storage**: JWT tokens and user session data
2. **Query Caching**: Frequently accessed prompts and analytics
3. **Rate Limiting**: Track API call counts per user/tenant
4. **Celery Broker**: Message queue for async tasks
5. **Temporary Data**: Test results awaiting persistence

Cache Strategy:

- TTL-based expiration (5-60 minutes depending on data type)
- Cache invalidation on data updates
- Lazy loading pattern (check cache → miss → fetch DB → populate cache)

3.2.5 Task Queue (Celery)

Purpose: Asynchronous job processing for time-consuming operations.

Task Types:

1. **LLM API Calls**: Send prompts to OpenAI/Anthropic and record results
2. **Batch Testing**: Run multiple prompt variants concurrently
3. **Analytics Aggregation**: Calculate daily/weekly/monthly stats

4. **Email Notifications**: Send alerts and digests
5. **GitHub Sync**: Import/export prompts from repositories
6. **Data Export**: Generate CSV/JSON exports of prompts/results

Configuration:

- RabbitMQ or Redis as message broker
- Multiple worker processes for parallelism
- Task retry logic with exponential backoff
- Result storage in Redis or database

3.2.6 External Integrations

OpenAI Integration:

- Models supported: GPT-4, GPT-4-Turbo, GPT-3.5-Turbo
- Features: Chat completions, streaming responses, token counting
- Error handling: Rate limit backoff, timeout management
- Cost tracking: Calculate costs based on token usage

Anthropic Integration:

- Models supported: Claude 3 Opus, Claude 3 Sonnet, Claude 3 Haiku
- Features: Message API, streaming, token counting
- Similar error handling and cost tracking as OpenAI

GitHub Integration:

- OAuth authentication for user accounts
- Repository webhooks for prompt sync
- Export prompts as markdown files to repo
- Import prompts from repo on commit

3.3 Data Flow

3.3.1 User Registration Flow

1. User submits registration form (email, password, org name)

↓

2. Frontend sends POST /api/auth/register

↓

3. Backend validates input (email format, password strength)

↓

4. Creates Tenant record with unique subdomain

↓

5. Creates User record linked to Tenant (role: org_admin)

↓

6. Generates JWT tokens (access + refresh)

↓

7. Returns tokens + user profile to frontend

↓

8. Frontend stores tokens in memory/localStorage

↓

9. Redirects user to dashboard

3.3.2 Prompt Creation Flow

1. User clicks "New Prompt" in UI

↓

2. Form captures title, description, content, tags

↓

3. Frontend sends POST /api/prompts/

↓

4. TenantMiddleware extracts tenant_id from JWT

↓

5. Backend validates and creates Prompt record

↓

6. Backend creates PromptVersion (v1) with content

↓

7. Sets Prompt.current_version to new version

↓

8. AuditLog records "prompt_created" action

↓

9. Returns Prompt + Version data to frontend

↓

10. Frontend navigates to prompt detail page

3.3.3 Prompt Testing Flow

1. User opens Test Sandbox, selects LLM model

↓

2. Optionally provides input variables

↓

3. Clicks "Run Test"

↓

4. Frontend sends POST /api/prompts/{id}/test

↓

5. Backend creates Celery task for LLM call

↓

6. Returns task_id immediately (async)

↓

7. Frontend polls GET /api/tasks/{task_id}/status

↓

8. Celery worker picks up task

↓

9. Worker calls OpenAI/Anthropic API

↓

10. Records start time, sends prompt, receives response

↓

11. Calculates metrics (tokens, cost, latency)

↓

12. Creates PromptTestRun record in DB

↓

13. Updates task status to "completed"

↓

14. Frontend receives completion, fetches results

↓

15. Displays output, metrics, and cost in UI

3.3.4 Version Comparison Flow

1. User navigates to Versions tab

↓

2. Frontend fetches GET /api/prompts/{id}/versions

↓

3. Backend queries PromptVersion table filtered by prompt_id and tenant_id

↓

4. Returns list of versions with metadata

↓

5. User selects two versions to compare

↓

6. Frontend performs client-side text diff

↓

7. Displays side-by-side or unified diff view

↓

8. User can click "Revert to this version"

↓

9. Backend creates new version copying old content

↓

10. Updates Prompt.current_version

↓

11. Returns updated prompt data

3.3.5 Analytics Data Flow

1. User navigates to Analytics dashboard

↓

2. Frontend sends GET /api/analytics/summary?period=30d

↓

3. Backend checks Redis cache for pre-computed data

↓

4. If cache miss, query PromptTestRun table

↓

5. Aggregate by day/week: count, avg latency, total tokens, total cost

↓

6. Group by prompt, team, user as needed

↓

7. Store results in Redis with 15-min TTL

↓

8. Return aggregated data to frontend

↓

9. Frontend renders charts using Recharts

↓

10. Daily Celery task pre-computes common aggregations

---

4. Detailed Design

4.1 Data Design / Database Schema

4.1.1 Core Tables

Table: tenants

```
CREATE TABLE tenants (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    subdomain VARCHAR(63) UNIQUE NOT NULL,
    plan_type VARCHAR(50) DEFAULT 'free',
    is_active BOOLEAN DEFAULT true,
    max_prompts INTEGER DEFAULT 50,
    max_users INTEGER DEFAULT 5,
    max_monthly_tests INTEGER DEFAULT 1000,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


CREATE INDEX idx_tenants_subdomain ON tenants(subdomain);
CREATE INDEX idx_tenants_active ON tenants(is_active);
```

Table: users

```
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id) ON DELETE CASCADE,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    first_name VARCHAR(150),
    last_name VARCHAR(150),
    role VARCHAR(50) DEFAULT 'editor',
    avatar VARCHAR(500),
    is_active BOOLEAN DEFAULT true,
    is_staff BOOLEAN DEFAULT false,
    is_superuser BOOLEAN DEFAULT false,
    last_login TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_tenant ON users(tenant_id);
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_tenant_role ON users(tenant_id, role);
```

Table: teams

```
CREATE TABLE teams (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    created_by UUID REFERENCES users(id) ON DELETE SET NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(tenant_id, name)
);
```

```
CREATE INDEX idx_teams_tenant ON teams(tenant_id);
```

Table: team_members (Many-to-Many)

```
CREATE TABLE team_members (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    team_id UUID REFERENCES teams(id) ON DELETE CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(team_id, user_id)
);

CREATE INDEX idx_team_members_team ON team_members(team_id);
CREATE INDEX idx_team_members_user ON team_members(user_id);
```

Table: prompts

```
CREATE TABLE prompts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id) ON DELETE CASCADE,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    team_id UUID REFERENCES teams(id) ON DELETE SET NULL,
    created_by UUID REFERENCES users(id) ON DELETE SET NULL,
    current_version_id UUID,  -- FK added after versions table
    tags JSONB DEFAULT '[]',
    category VARCHAR(100),
    is_archived BOOLEAN DEFAULT false,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_prompts_tenant ON prompts(tenant_id, updated_at DESC);
CREATE INDEX idx_prompts_tenant_archived ON prompts(tenant_id, is_archived);
```

```
CREATE INDEX idx_prompts_team ON prompts(team_id);
CREATE INDEX idx_prompts_created_by ON prompts(created_by);
CREATE INDEX idx_prompts_tags ON prompts USING GIN(tags);
```

Table: prompt_versions

```
CREATE TABLE prompt_versions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    prompt_id UUID REFERENCES prompts(id) ON DELETE CASCADE,
    version_number INTEGER NOT NULL,
    content TEXT NOT NULL,
    status VARCHAR(50) DEFAULT 'draft',
    diff_summary TEXT,
    parent_version_id UUID REFERENCES prompt_versions(id) ON DELETE SET NULL,
    model_config JSONB DEFAULT '{}',
    created_by UUID REFERENCES users(id) ON DELETE SET NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(prompt_id, version_number)
);

CREATE INDEX idx_prompt_versions_prompt ON prompt_versions(prompt_id, version_number
DESC);
CREATE INDEX idx_prompt_versions_status ON prompt_versions(status);

-- Add FK to prompts table after versions exists
ALTER TABLE prompts ADD CONSTRAINT fk_prompts_current_version
    FOREIGN KEY (current_version_id) REFERENCES prompt_versions(id) ON DELETE SET NULL;
```

Table: prompt_test_runs

```
CREATE TABLE prompt_test_runs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    prompt_version_id UUID REFERENCES prompt_versions(id) ON DELETE CASCADE,
    llm_provider VARCHAR(50) NOT NULL,
    llm_model VARCHAR(100) NOT NULL,
```

```
    input_variables JSONB DEFAULT '{}',
    input_text TEXT NOT NULL,
    output_text TEXT,
    response_time_ms INTEGER,
    token_count_input INTEGER,
    token_count_output INTEGER,
    cost_usd DECIMAL(10,6),
    metrics JSONB DEFAULT '{}',
    status VARCHAR(50) DEFAULT 'success',
    error_message TEXT,
    run_by UUID REFERENCES users(id) ON DELETE SET NULL,
    run_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_test_runs_version ON prompt_test_runs(prompt_version_id, run_at DESC);
CREATE INDEX idx_test_runs_user ON prompt_test_runs(run_by, run_at DESC);
CREATE INDEX idx_test_runs_status ON prompt_test_runs(status);
CREATE INDEX idx_test_runs_run_at ON prompt_test_runs(run_at DESC);
```

Table: comments

```
CREATE TABLE comments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    prompt_version_id UUID REFERENCES prompt_versions(id) ON DELETE CASCADE,
    author_id UUID REFERENCES users(id) ON DELETE SET NULL,
    text TEXT NOT NULL,
    parent_id UUID REFERENCES comments(id) ON DELETE CASCADE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_comments_version ON comments(prompt_version_id, created_at);
CREATE INDEX idx_comments_author ON comments(author_id);
```

```
CREATE INDEX idx_comments_parent ON comments(parent_id);
```

Table: audit_logs

```
CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID REFERENCES tenants(id) ON DELETE CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    action VARCHAR(100) NOT NULL,
    target_type VARCHAR(50) NOT NULL,
    target_id UUID NOT NULL,
    details JSONB DEFAULT '{}',
    ip_address INET,
    user_agent TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_audit_logs_tenant ON audit_logs(tenant_id, timestamp DESC);
CREATE INDEX idx_audit_logs_user ON audit_logs(user_id, timestamp DESC);
CREATE INDEX idx_audit_logs_target ON audit_logs(target_type, target_id);
CREATE INDEX idx_audit_logs_timestamp ON audit_logs(timestamp DESC);
```

4.1.2 Relationships

```
tenants 1————————* users
tenants 1————————* teams
tenants 1————————* prompts
tenants 1————————* audit_logs

users *————————* teams (via team_members)
users 1————————* prompts (created_by)
users 1————————* prompt_versions (created_by)
users 1————————* prompt_test_runs (run_by)
users 1————————* comments (author_id)
```

prompts 1————* prompt_versions

prompts *————1 prompt_versions (current_version)

prompts *————1 teams

prompt_versions 1————* prompt_test_runs

prompt_versions 1————* comments

prompt_versions 1————* prompt_versions (parent/child)

comments 1————* comments (parent/replies)

4.1.3 Data Integrity Rules

1. **Tenant Isolation**: Every query MUST filter by tenant_id except for system admin operations
2. **Cascade Deletes**: Deleting a tenant removes all associated data
3. **Soft Deletes**: Prompts use is_archived flag; users use is_active flag
4. **Version Immutability**: Once created, prompt_versions records are never modified
5. **Audit Trail**: All significant actions (create, update, delete, deploy) logged to audit_logs

4.2 API Design

4.2.1 API Structure

Base URL: https://api.promptops.io/v1/

Authentication: JWT Bearer token in Authorization header

Authorization: Bearer <access_token>

Response Format: JSON with consistent structure

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "page": 1,
    "per_page": 20,
    "total": 150
  }
}
```

Error Format:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid input data",
    "details": {
      "email": ["This field is required"]
    }
  }
}
```


4.2.2 Authentication Endpoints

POST /auth/register

Request:


```
{
  "email": "user@example.com",
  "password": "SecurePass123!",
  "password_confirm": "SecurePass123!",
  "first_name": "John",
  "last_name": "Doe",
  "tenant_name": "Acme Corp",
  "tenant_subdomain": "acme"
}
```


Response: 201 Created


```
{
  "success": true,
  "data": {
    "user": {
      "id": "uuid",
      "email": "user@example.com",
      "first_name": "John",
```

```
    "last_name": "Doe",
    "role": "org_admin"
   },
   "tenant": {
    "id": "uuid",
    "name": "Acme Corp",
    "subdomain": "acme"
   },
   "tokens": {
    "access": "eyJ...",
    "refresh": "eyJ..."
   }
  }
}
```


POST /auth/login

Request:


```
{
  "email": "user@example.com",
  "password": "SecurePass123!"
}
```


Response: 200 OK


```
{
  "success": true,
  "data": {
    "user": { ... },
    "tokens": {
      "access": "eyJ...",
      "refresh": "eyJ..."
    }
  }
```

}

POST /auth/refresh

Request:

{

"refresh": "eyJ..."

}

Response: 200 OK

{

"success": true,

"data": {

"access": "eyJ...",

"refresh": "eyJ..."

}

}

POST /auth/logout

Request: (JWT token in header)

Response: 204 No Content

4.2.3 Prompt Endpoints

GET /prompts

Query Params:

- page=1
- per_page=20
- search=keyword
- category=string
- tags=tag1,tag2
- team_id=uuid
- is_archived=false

Response: 200 OK

{

```
 "success": true,
 "data": [
  {
   "id": "uuid",
   "title": "Customer Support Bot Prompt",
   "description": "Handles common customer inquiries",
   "current_version": {
    "id": "uuid",
    "version_number": 3,
    "content": "You are a helpful...",
    "status": "deployed"
   },
   "tags": ["support", "chatbot"],
   "category": "customer-service",
   "version_count": 3,
   "latest_test_date": "2024-03-15T10:30:00Z",
   "created_by": {
    "id": "uuid",
    "name": "John Doe"
   },
   "created_at": "2024-03-01T08:00:00Z",
   "updated_at": "2024-03-15T10:30:00Z"
  }
 ],
 "meta": {
  "page": 1,
  "per_page": 20,
  "total": 45
 }
}
```


POST /prompts

Request:

```
{
  "title": "New Marketing Copy Generator",
  "description": "Creates engaging product descriptions",
  "content": "You are an expert copywriter...",
  "tags": ["marketing", "copywriting"],
  "category": "content-generation",
  "team_id": "uuid",
  "model_config": {
    "temperature": 0.7,
    "max_tokens": 500
  }
}
```

Response: 201 Created

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "title": "New Marketing Copy Generator",
    "current_version": {
      "id": "uuid",
      "version_number": 1,
      "content": "You are an expert copywriter...",
      "status": "draft"
    },
    ...
  }
}
```

GET /prompts/:id

Response: 200 OK

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "title": "...",
    "description": "...",
    "current_version": { ... },
    "versions": [
      {
        "id": "uuid",
        "version_number": 3,
        "content": "...",
        "status": "deployed",
        "created_by": { ... },
        "created_at": "..."
      },
      ...
    ],
    "test_run_count": 47,
    "avg_response_time_ms": 1250,
    "total_cost_usd": 12.45
  }
}
```


PATCH /prompts/:id

Request:


```
{
  "title": "Updated Title",
  "content": "Updated prompt content...",
  "tags": ["new", "tags"]
}
```

Response: 200 OK

(Creates new version if content changed)


```
{
  "success": true,
  "data": {
    "id": "uuid",
    "current_version": {
      "version_number": 4,  // New version created
      "content": "Updated prompt content..."
    },
    ...
  }
}
```


DELETE /prompts/:id

Response: 204 No Content

(Soft delete - sets is_archived=true)

4.2.4 Version Endpoints

GET /prompts/:id/versions

Response: 200 OK


```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "version_number": 3,
      "content": "...",
      "status": "deployed",
      "diff_summary": "Updated temperature parameter",
      "created_by": { ... },
      "created_at": "..."
    },
```

```
    ...
  ]
}
```

GET /prompts/:id/versions/:version_id

Response: 200 OK

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "version_number": 2,
    "content": "Full content here...",
    "model_config": { ... },
    "test_runs": [ ... ]
  }
}
```

POST /prompts/:id/revert

Request:

```
{
  "version_id": "uuid"
}
```

Response: 200 OK

(Creates new version with old content)

4.2.5 Testing Endpoints

POST /prompts/:id/test

Request:

```
{
  "version_id": "uuid",  // Optional, defaults to current
  "llm_provider": "openai",
  "llm_model": "gpt-4",
  "input_variables": {
```

```
    "product_name": "Widget Pro",
    "features": ["durable", "efficient"]
  }
}
```

Response: 202 Accepted

```
{
  "success": true,
  "data": {
    "task_id": "celery-task-uuid"
  }
}
```

GET /tasks/:task_id

Response: 200 OK

```
{
  "success": true,
  "data": {
    "task_id": "uuid",
    "status": "completed",  // pending, running, completed, failed
    "result": {
      "id": "test-run-uuid",
      "output_text": "Introducing Widget Pro...",
      "response_time_ms": 1250,
      "token_count_input": 45,
      "token_count_output": 120,
      "cost_usd": 0.0025
    }
  }
}
```

GET /prompts/:id/test-history

Query Params: page, per_page, llm_provider, status

Response: 200 OK


```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "llm_provider": "openai",
      "llm_model": "gpt-4",
      "output_text": "...",
      "response_time_ms": 1250,
      "cost_usd": 0.0025,
      "status": "success",
      "run_at": "..."
    },
    ...
  ]
}
```


4.2.6 Analytics Endpoints

GET /analytics/summary

Query Params:

- period=7d,30d,90d,all
- team_id=uuid

Response: 200 OK


```
{
  "success": true,
  "data": {
    "total_prompts": 45,
    "total_test_runs": 3250,
    "total_cost_usd": 456.78,
```

```
    "avg_response_time_ms": 1350,
    "test_runs_by_day": [
      {"date": "2024-03-01", "count": 150, "cost": 15.23},
      ...
    ],
    "top_prompts": [
      {"prompt_id": "uuid", "title": "...", "test_count": 450},
      ...
    ],
    "cost_by_model": [
      {"model": "gpt-4", "cost": 300.00},
      {"model": "claude-3-opus", "cost": 156.78}
    ]
  }
}
```

GET /analytics/prompts/:id

Response: 200 OK

```
{
  "success": true,
  "data": {
    "prompt_id": "uuid",
    "total_test_runs": 450,
    "total_cost_usd": 45.67,
    "avg_response_time_ms": 1200,
    "success_rate": 0.98,
    "usage_over_time": [ ... ],
    "model_distribution": [ ... ]
  }
}
```

4.2.7 Team Endpoints

GET /teams POST /teams GET /teams/:id PATCH /teams/:id DELETE /teams/:id POST /teams/:id/members - Add member **DELETE /teams/:id/members/:user_id** - Remove member

4.2.8 User Management Endpoints

GET /users - List users in tenant **GET /users/:id** - Get user details **PATCH /users/:id** - Update user (profile, role) **DELETE /users/:id** - Deactivate user

4.2.9 Integration Endpoints

POST /integrations/github/connect - OAuth flow **GET /integrations/github/repos** - List connected repos **POST /integrations/github/sync** - Trigger sync **POST /integrations/github/webhook** - Receive GitHub webhooks

4.3 User Interface (UI/UX)

4.3.1 Design Principles

1. **Simplicity First**: Clean, uncluttered interface focused on core workflows
2. **Responsive Design**: Works seamlessly on desktop (1920x1080) down to tablet (768px)
3. **Accessibility**: WCAG 2.1 AA compliance, keyboard navigation, screen reader support
4. **Fast Feedback**: Loading states, optimistic updates, error messages
5. **Consistent Patterns**: Reusable components, consistent spacing/colors

4.3.2 Color Scheme

Primary: #3B82F6 (Blue-500) - CTAs, links

Secondary: #8B5CF6 (Purple-500) - Highlights

Success: #10B981 (Green-500) - Success states

Warning: #F59E0B (Amber-500) - Warnings

Error: #EF4444 (Red-500) - Errors

Gray: #6B7280 (Gray-500) - Text, borders

Background: #F9FAFB (Gray-50)

Card: #FFFFFF

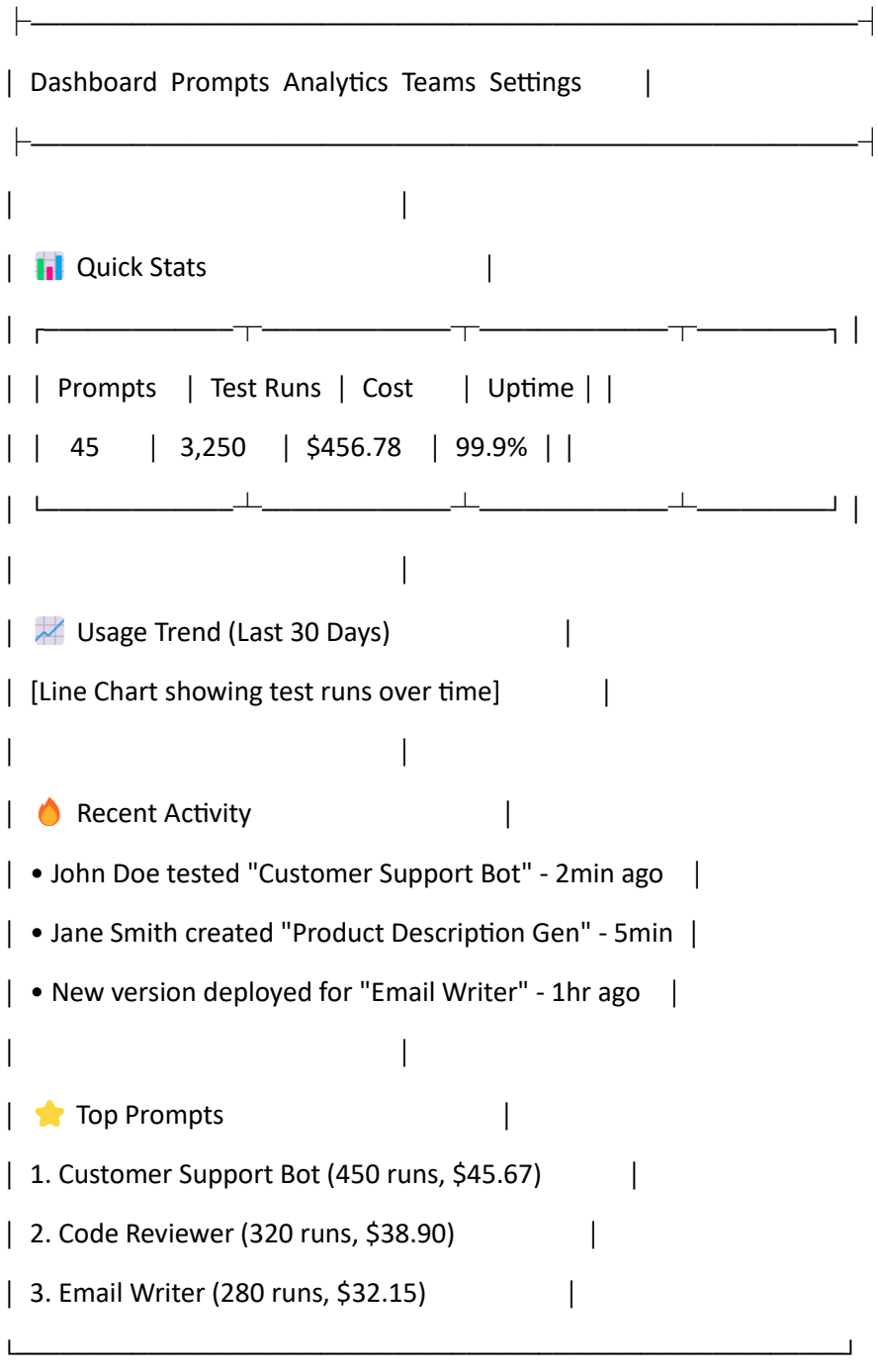
Text Primary: #111827 (Gray-900)

Text Secondary: #6B7280 (Gray-500)

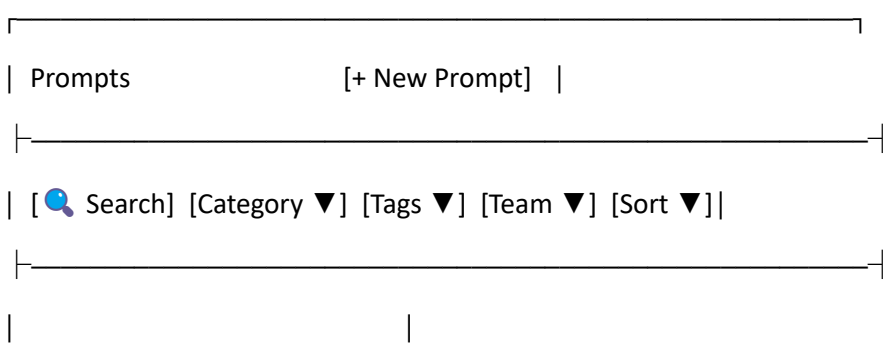
4.3.3 Key Screens

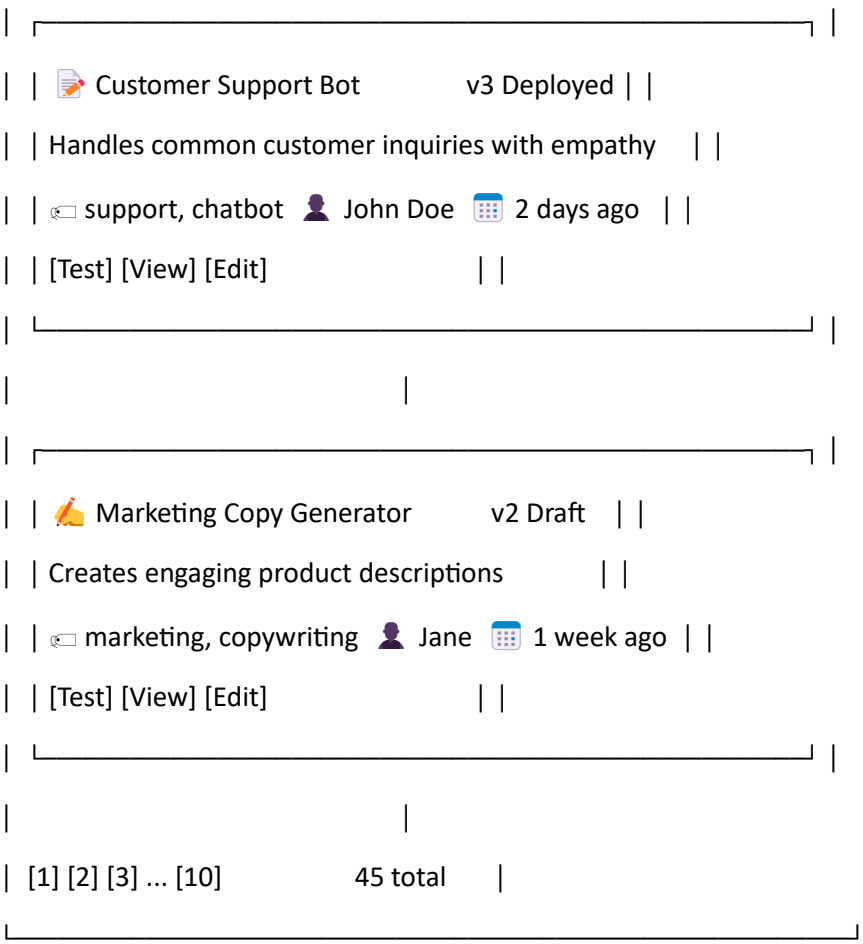
1. Dashboard / Home

```
┌──────────────────────────────────────────────────────────┐
| PromptOps Logo  [Search...]    [+ New]  [@User ▼] |
```

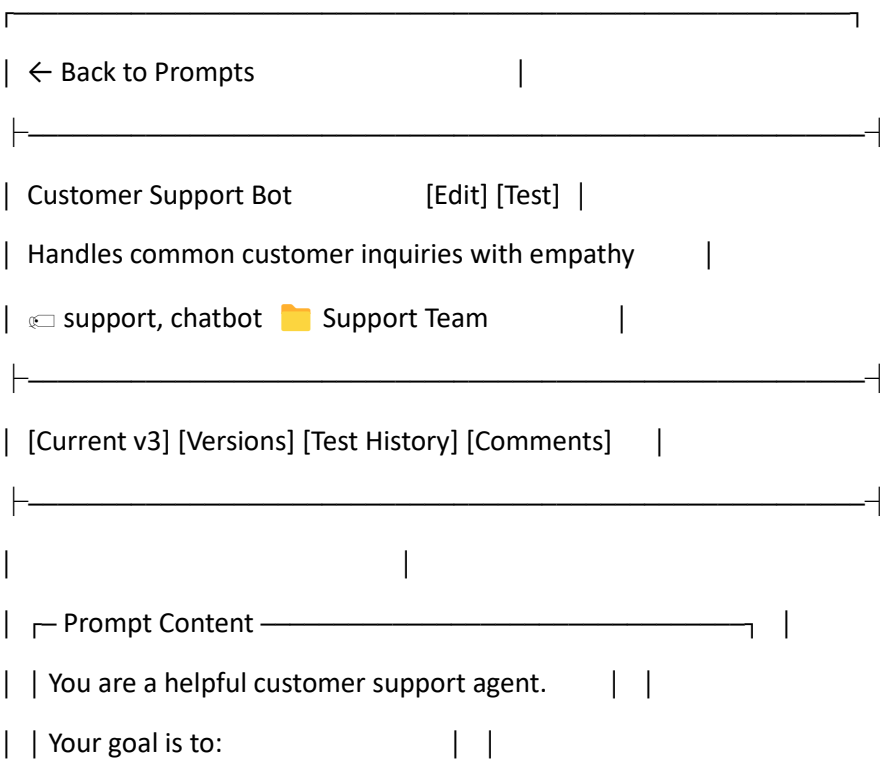


2. Prompt Library





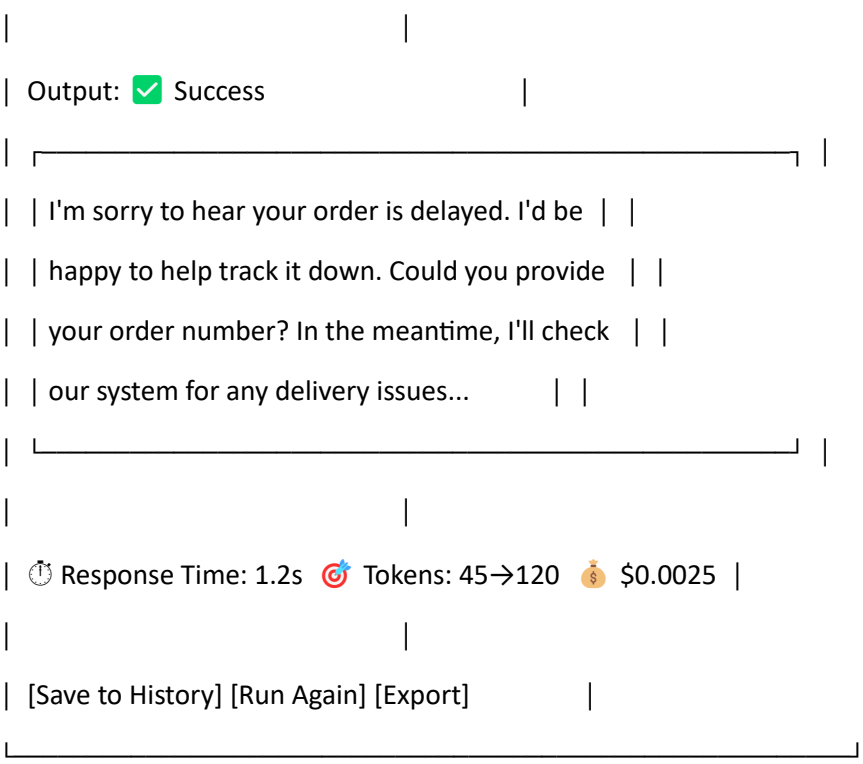
3. Prompt Detail View



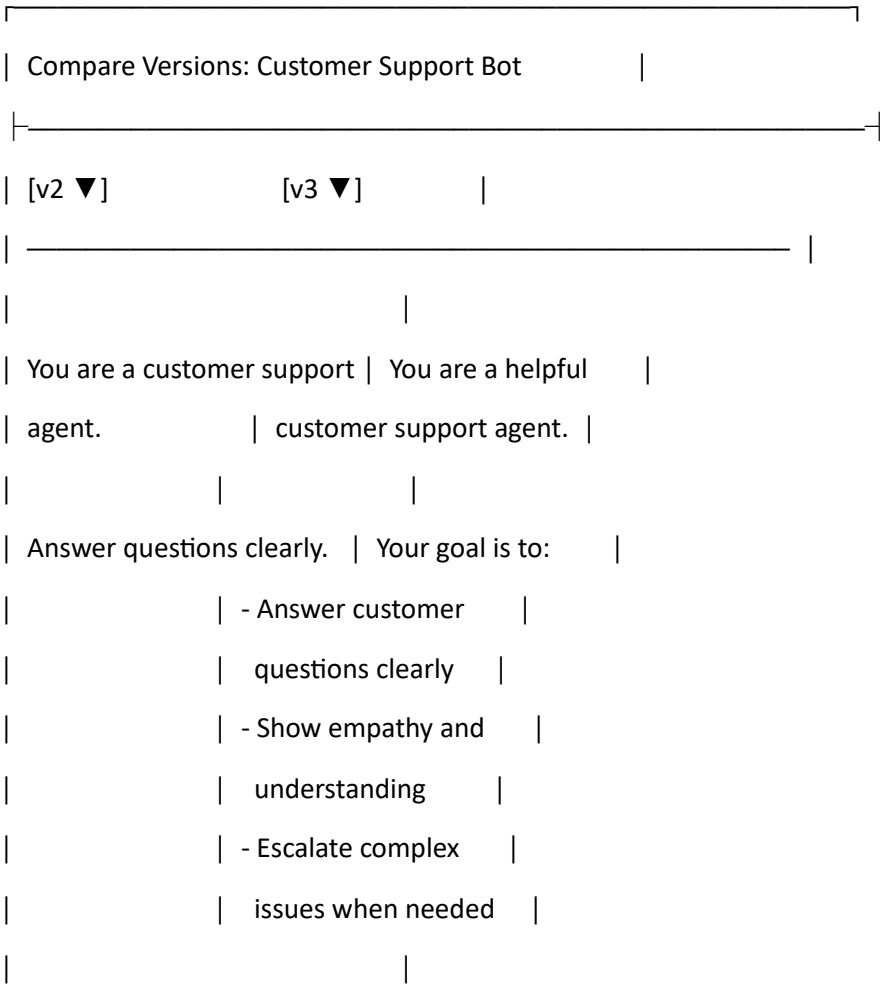
```
| | - Answer customer questions clearly          | |
| | - Show empathy and understanding             | |
| | - Escalate complex issues when needed        | |
| |                                              | |
| | Customer Query: {{customer_message}}         | |
| └──────────────────────────────────────────────┘ |
|                                                  |
| ⚙️ Model Configuration                           |
| Provider: OpenAI  Model: gpt-4  Temp: 0.7  Max: 500 |
|                                                  |
| 📊 Stats                                         |
| Test Runs: 450  Avg Response: 1.2s  Total Cost: $45.67 |
└──────────────────────────────────────────────────┘
```

4. Testing Sandbox

```
┌──────────────────────────────────────────────────┐
| Test: Customer Support Bot v3                    |
├──────────────────────────────────────────────────┤
| Provider: [OpenAI ▼]  Model: [gpt-4 ▼]           |
|                                                  |
| Input Variables:                                 |
| ┌──────────────────────────────────────────────┐ |
| | customer_message:                          | |
| | ┌──────────────────────────────────────────┐ | |
| | | I haven't received my order yet. It's been | | |
| | | 5 days since I placed it.                | | |
| | └──────────────────────────────────────────┘ | |
| └──────────────────────────────────────────────┘ |
|                                                  |
| [▶ Run Test]                                     |
|                                                  |
| ──────────────────────────────────────────────── |
```



5. Version Comparison



```
| Customer Query: {{message}} |  Customer Query:        |
|                             | {{customer_message}}    |
|                                       |
| [← Revert to v2]              [Use v3 →]       |
└──────────────────────────────────────────────────────┘
```

6. Analytics Dashboard

```
┌──────────────────────────────────────────────────────┐
| Analytics              [7d ▼] [Team: All ▼]      |
├──────────────────────────────────────────────────────┤
|                                       |
| 📊 Overview                           |
| ┌──────────┬──────────┬──────────┬──────────┐  |
| | Test Runs | Avg Time  | Cost     | Success   |  |
| |  3,250   |  1.3s    | $456.78  |  98.5%    |  |
| | +12% ↑   | -0.1s ↓  | +$45 ↑   | +0.2% ↑   |  |
| └──────────┴──────────┴──────────┴──────────┘  |
|                                       |
| 📈 Test Runs Over Time                |
| [Line chart: X-axis dates, Y-axis count]      |
|                                       |
| 💰 Cost Breakdown by Model            |
| [Pie chart: GPT-4 65%, Claude


| [Pie chart: GPT-4 65%, Claude-3 25%, GPT-3.5 10%]     |
|                                       |
| 🔥 Most Used Prompts                  |
| ┌──────────────────────────────────────────────┐ |
| | 1. Customer Support Bot  450 runs  $45.67  1.2s  | |
| | 2. Code Reviewer         320 runs  $38.90  1.5s  | |
| | 3. Email Writer          280 runs  $32.15  0.9s  | |
| └──────────────────────────────────────────────┘ |
```

```
|                                                                    |
| [Export CSV] [Export JSON]                                         |
└────────────────────────────────────────────────────────────────────┘
```

4.3.4 Component Library

Reusable Components:

1. **Button** - Primary, Secondary, Danger, Text variants
2. **Input** - Text, Textarea, Select, Checkbox
3. **Card** - Container with shadow and padding
4. **Modal** - Overlay dialog for confirmations/forms
5. **Table** - Sortable, paginated data table
6. **Tabs** - Horizontal navigation between views
7. **Badge** - Status indicators (Draft, Deployed, etc.)
8. **Avatar** - User profile pictures with fallback initials
9. **LoadingSpinner** - Loading state indicator
10. **Toast** - Notification messages (success, error, info)
11. **CodeEditor** - Syntax-highlighted prompt editor
12. **Chart** - Line, Bar, Pie chart wrappers

4.3.5 User Flows

New User Onboarding:

1. User lands on marketing site → Clicks "Sign Up"
2. Registration form (email, password, org name)
3. Email verification (optional for MVP)
4. Welcome screen with quick tour
5. "Create First Prompt" guided flow
6. Dashboard with sample data/tips

Creating a Prompt:

1. Click "+ New Prompt" from anywhere
2. Modal/page with form (title, description, content)
3. Optionally add tags, category, team
4. Click "Create" → Redirects to prompt detail
5. Toast: "Prompt created successfully"

Testing a Prompt:

1. From prompt detail, click "Test"
2. Testing sandbox opens
3. Select LLM provider and model
4. Fill input variables (if any)
5. Click "Run Test"
6. Loading spinner while API call in progress
7. Results display with output, metrics, cost
8. Option to save, run again, or export

Comparing Versions:

1. Navigate to "Versions" tab on prompt detail
2. List of all versions shown with dates
3. Click "Compare" button between two versions
4. Side-by-side diff view
5. Option to revert to older version
6. Confirmation modal: "Create new version from v2?"
7. New version created with old content

4.4 Technology Stack

4.4.1 Backend Stack

Core Framework:

- **Django 5.0**: Full-featured web framework
    - Rationale: Rapid development, built-in admin, ORM, security features
    - Alternatives considered: FastAPI (chose Django for built-in features)

API Layer:

- **Django REST Framework 3.14**: RESTful API toolkit
    - Rationale: Mature, well-documented, integrates seamlessly with Django
    - Serializers, ViewSets, authentication built-in

Authentication:

- **djangorestframework-simplejwt**: JWT token management
    - Rationale: Stateless auth, works well with SPAs
    - Access + Refresh token pattern

Database:

- **PostgreSQL 16**: Primary relational database
    - Rationale: ACID compliance, JSON support, full-text search, reliability
    - Alternatives: MySQL (chose Postgres for advanced features)

Caching:

- **Redis 7**: In-memory data store
    - Rationale: Fast, supports caching + queuing, mature
    - Used for: Sessions, rate limiting, Celery broker

Task Queue:

- **Celery 5.3**: Distributed task queue
    - Rationale: Standard for async tasks in Django, reliable
    - Used for: LLM API calls, analytics aggregation, email sending

LLM Client Libraries:

- **openai 1.12.0**: OpenAI API client
- **anthropic 0.18.0**: Anthropic (Claude) API client

Additional Libraries:

- **python-decouple**: Environment variable management
- **django-cors-headers**: CORS handling for SPA
- **django-filter**: Advanced query filtering
- **drf-spectacular**: OpenAPI schema generation
- **Pillow**: Image processing for avatars
- **requests**: HTTP library for GitHub API

4.4.2 Frontend Stack

Core Framework:

- **React 18**: UI library
    - Rationale: Component-based, large ecosystem, job market demand
    - Alternatives: Vue.js (chose React for ecosystem size)

State Management:

- **TanStack Query (React Query) 5**: Server state management
    - Rationale: Handles caching, background updates, optimistic updates automatically
    - Reduces boilerplate compared to Redux

Routing:

- **React Router 6**: Client-side routing
    - Rationale: Standard routing library, nested routes, lazy loading

Styling:

- **Tailwind CSS 3**: Utility-first CSS framework
    - Rationale: Rapid development, consistent design, small bundle size
    - Responsive utilities built-in

UI Components:

- **Headless UI**: Unstyled accessible components (modals, dropdowns)
    - Rationale: Accessibility built-in, full styling control
- **Radix UI**: Complex components (select, tabs, dialog)

Charts:

- **Recharts 2**: Data visualization
    - Rationale: Composable, responsive, React-native
    - Alternatives: Chart.js (chose Recharts for React integration)

Code Editor:

- **Monaco Editor**: VS Code-based editor
    - Rationale: Syntax highlighting, diff viewer, mature
    - Used for prompt editing and version comparison

HTTP Client:

- **Axios 1.6**: Promise-based HTTP client
    - Rationale: Interceptors for auth tokens, request/response transformation

Build Tool:

- **Vite 5**: Frontend build tool
    - Rationale: Fast dev server, optimized production builds
    - Alternatives: Create React App (chose Vite for speed)

Additional Libraries:

- **date-fns**: Date formatting and manipulation
- **react-hot-toast**: Toast notifications
- **react-icons**: Icon library
- **clsx**: Conditional className utility

4.4.3 DevOps & Infrastructure

Containerization:

- **Docker 24**: Container platform
- **Docker Compose**: Local multi-container orchestration

Hosting (Initial):

- **Railway.app** or **Render.com**: Platform-as-a-Service
    - Rationale: Free tier available, simple deployment, PostgreSQL included
    - Alternatives: Heroku, AWS (chose Railway for cost/simplicity)

CI/CD:

- **GitHub Actions**: Automated testing and deployment
    - Rationale: Integrated with GitHub, free for public repos
    - Workflows: Test on PR, Deploy on merge to main

Monitoring (Future):

- **Sentry**: Error tracking
- **PostHog**: Product analytics
- **Prometheus + Grafana**: System metrics (when self-hosting)

Domain & DNS:

- **Namecheap** or **Cloudflare**: Domain registration and DNS
- **Cloudflare**: CDN and DDoS protection (free tier)

4.4.4 Development Tools

Version Control:

- **Git** + **GitHub**: Source control and collaboration
- **GitHub Projects**: Issue tracking and kanban boards

Code Quality:

- **Black**: Python code formatter
- **Flake8**: Python linter
- **isort**: Import sorting
- **ESLint**: JavaScript linter
- **Prettier**: JavaScript formatter

Testing:

- **pytest**: Python testing framework

- **pytest-django**: Django test utilities
- **pytest-cov**: Code coverage
- **Jest**: JavaScript unit testing
- **React Testing Library**: React component testing

Database Management:

- **pgAdmin**: PostgreSQL GUI (optional)
- **Django Admin**: Built-in admin interface

API Testing:

- **Postman** or **Insomnia**: Manual API testing
- **OpenAPI/Swagger UI**: Auto-generated API docs

Documentation:

- **MkDocs**: Documentation site generator
- **Docusaurus**: Alternative documentation platform

4.5 Error Handling and Security

4.5.1 Error Handling Strategy

Backend Error Handling:

1. **Validation Errors** (400 Bad Request)

```
# Caught by DRF serializers
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid input data",
    "details": {
      "email": ["Enter a valid email address"],
      "password": ["This field is required"]
    }
  }
}
```

2. **Authentication Errors** (401 Unauthorized)

```
# Invalid or expired tokens
{
```

```
  "error": {
    "code": "AUTHENTICATION_FAILED",
    "message": "Invalid or expired token"
  }
}
```

3. **Permission Errors** (403 Forbidden)

```
# User lacks permission for action
{
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "You don't have permission to perform this action"
  }
}
```

4. **Not Found Errors** (404)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Prompt not found"
  }
}
```

5. **Rate Limit Errors** (429 Too Many Requests)

```
{
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Too many requests. Try again in 60 seconds",
    "retry_after": 60
  }
}
```

6. **Server Errors** (500 Internal Server Error)

```
# Logged to Sentry, generic message to user
{
```

```
  "error": {
    "code": "INTERNAL_ERROR",
    "message": "An unexpected error occurred. Our team has been notified.",
    "error_id": "uuid"  # For support reference
  }
}
```

Frontend Error Handling:

1. **Network Errors**: Display retry button, check internet connection
2. **Validation Errors**: Show inline field errors, prevent form submission
3. **API Errors**: Display toast notifications with error messages
4. **Loading States**: Show skeletons/spinners during async operations
5. **Fallback UI**: Error boundaries to catch React crashes

LLM API Error Handling:

1. **Rate Limiting**: Exponential backoff with jitter, queue requests
2. **Timeouts**: 30-second timeout, retry up to 3 times
3. **API Errors**: Log error, return user-friendly message
4. **Cost Limits**: Reject requests if monthly budget exceeded

Error Logging:

- All errors logged with context (user_id, tenant_id, request_id)
- Critical errors trigger alerts (email/Slack)
- Error aggregation in Sentry for trend analysis

4.5.2 Security Measures

Authentication Security:

1. **Password Requirements:**
    - Minimum 8 characters
    - At least one uppercase, lowercase, number
    - Django's built-in password validators
    - Bcrypt hashing (Django default)
2. **JWT Token Security:**
    - Short-lived access tokens (60 minutes)
    - Longer refresh tokens (24 hours)

- Tokens include tenant_id claim
- HTTPS-only transmission
- Optional: Token blacklist on logout

3. **Session Security:**
    - Secure, HttpOnly, SameSite cookies
    - CSRF protection enabled
    - Session timeout after 24 hours inactivity

Data Security:

1. **Tenant Isolation:**
    - Every query filters by tenant_id (enforced at ORM level)
    - Middleware injects tenant context
    - Database-level row security (future enhancement)
2. **Encryption:**
    - TLS/HTTPS for all traffic (enforced)
    - Database connections encrypted
    - API keys encrypted at rest (Fernet symmetric encryption)
    - User passwords hashed with Bcrypt
3. **Input Validation:**
    - All inputs validated via DRF serializers
    - SQL injection prevented by ORM parameterized queries
    - XSS protection via React's automatic escaping
    - CSRF tokens for state-changing operations

API Security:

1. **Rate Limiting:**
    - 100 requests per minute per user
    - 1000 requests per hour per tenant
    - Redis-based token bucket algorithm
2. **CORS Policy:**
    - Whitelist specific frontend domains
    - Credentials allowed for authenticated requests
    - No wildcard origins in production

3. **Request Size Limits:**
    - Max request body: 10MB
    - Max prompt content: 100KB
    - File uploads: 5MB per file

Infrastructure Security:

1. **Secret Management:**
    - Environment variables for all secrets
    - Never commit secrets to Git (.gitignore enforcement)
    - Rotate API keys quarterly
2. **Database Security:**
    - Separate database user for application
    - Minimal permissions (no DROP, TRUNCATE)
    - Regular backups (daily automated)
    - Connection from application server only
3. **Dependency Management:**
    - Regular dependency updates (monthly)
    - Automated vulnerability scanning (Dependabot)
    - Pin exact versions in production

Compliance & Privacy:

1. **Data Retention:**
    - Audit logs retained for 1 year
    - Test runs retained for 90 days (configurable)
    - Deleted users: Data anonymized, not hard-deleted
2. **GDPR Compliance:**
    - Data export API endpoint
    - Account deletion with data purge option
    - Clear privacy policy and terms of service
3. **Audit Trail:**
    - Log all CRUD operations
    - Track IP address and user agent
    - Immutable audit log (append-only)

Security Testing:

1. **Automated Scanning:**
    - OWASP ZAP for vulnerability scanning
    - Bandit for Python security linting
    - npm audit for JavaScript dependencies
2. **Manual Testing:**
    - Penetration testing before public launch
    - Security code review of critical paths
    - Regular security audits (quarterly)

---

5. Implementation Plan and Timeline

5.1 Milestones

This implementation plan assumes **solo development** with **15-20 hours per week** of focused work. Total estimated time: **16-20 weeks (4-5 months)**.

Phase 1: Foundation (Weeks 1-4)

Week 1-2: Project Setup & Core Models

- ✅ Initialize Git repository
- ✅ Set up Django project structure
- ✅ Configure PostgreSQL + Redis via Docker Compose
- ✅ Create core models (Tenant, User, Team)
- ✅ Set up Django admin for data inspection
- ✅ Write initial database migrations
- ✅ Configure environment variables

Deliverable: Working Django app with database models

Week 3-4: Authentication & User Management

- Implement JWT authentication (login, register, refresh)
- Create user registration with tenant creation
- Build user profile API endpoints
- Add password reset functionality
- Write unit tests for auth flows
- Create basic Django REST Framework API structure

Deliverable: Complete authentication API with tests

Phase 2: Core Prompt Management (Weeks 5-8)

Week 5-6: Prompt CRUD & Versioning

- Create Prompt and PromptVersion models
- Implement prompt CRUD API endpoints
- Add automatic version creation on edit
- Build version listing and detail endpoints
- Implement version comparison logic
- Add search and filtering
- Write comprehensive tests

Deliverable: Full prompt management API

Week 7-8: Frontend Foundation

- Set up React + Vite project
- Configure Tailwind CSS
- Create authentication pages (login, register)
- Build layout components (sidebar, header)
- Implement API client with Axios + React Query
- Create dashboard with placeholder data
- Add routing with React Router

Deliverable: Working frontend with auth and navigation

Phase 3: Testing & LLM Integration (Weeks 9-11)

Week 9: LLM Integration

- Implement OpenAI client service
- Implement Anthropic client service
- Create test run model and API
- Build Celery tasks for async LLM calls
- Add token counting and cost calculation
- Implement error handling and retries

Deliverable: Backend LLM testing functionality

Week 10-11: Testing Sandbox UI

- Build testing sandbox page

- Create LLM provider/model selectors
- Add input variable form
- Display loading states during API calls
- Show test results with metrics
- Create test history view
- Add export functionality

Deliverable: Complete testing feature

Phase 4: Collaboration & Analytics (Weeks 12-14)

Week 12: Comments & Teams

- Implement comment model and API
- Create team CRUD endpoints
- Add team member management
- Build comment UI components
- Add threaded reply functionality
- Implement real-time updates (polling)

Deliverable: Collaboration features

Week 13-14: Analytics Dashboard

- Create analytics aggregation logic
- Build summary statistics endpoint
- Implement cost breakdown calculations
- Add usage trend endpoints
- Create analytics dashboard UI
- Build charts with Recharts
- Add date range filtering
- Implement data export (CSV/JSON)

Deliverable: Analytics dashboard

Phase 5: Polish & Launch Prep (Weeks 15-16)

Week 15: UI/UX Polish

- Responsive design improvements
- Add loading skeletons
- Implement toast notifications

- Error message improvements
- Keyboard shortcuts
- Accessibility audit and fixes
- Performance optimization (lazy loading, code splitting)

Deliverable: Polished user experience

Week 16: Documentation & Deployment

- Write user documentation
- Create API documentation (OpenAPI/Swagger)
- Set up CI/CD pipeline (GitHub Actions)
- Deploy to Railway/Render
- Configure domain and SSL
- Set up error monitoring (Sentry)
- Create backup scripts
- Write README and contributing guide

Deliverable: Deployed production application

Phase 6: Post-Launch (Weeks 17-20)

Week 17-18: GitHub Integration

- Implement GitHub OAuth flow
- Add repository connection API
- Build webhook receiver
- Create prompt export to repo
- Add import from repo functionality
- Build integration UI

Deliverable: GitHub integration

Week 19-20: Advanced Features

- Add basic pipeline functionality
- Implement prompt templates/variables
- Add bulk operations
- Email notifications for key events
- Performance monitoring dashboard
- User onboarding flow improvements

Deliverable: Enhanced feature set

5.2 Testing and Quality Assurance

5.2.1 Testing Strategy

Unit Tests (70% coverage minimum)

Backend (pytest + pytest-django):

```
# Example test structure
tests/
├── accounts/
│   ├── test_models.py
│   ├── test_serializers.py
│   ├── test_views.py
│   └── test_permissions.py
├── prompts/
│   ├── test_models.py
│   ├── test_version_creation.py
│   ├── test_api_endpoints.py
│   └── test_llm_integration.py
├── analytics/
│   ├── test_aggregations.py
│   └── test_cost_calculations.py
└── fixtures/
    └── sample_data.py
```

Frontend (Jest + React Testing Library):

```
// Example test structure
src/
├── components/
│   ├── Button.test.jsx
│   ├── PromptCard.test.jsx
│   └── TestSandbox.test.jsx
├── hooks/
```

```
|   └── usePrompts.test.js
├── utils/
|   └── api.test.js
└── pages/
    └── Dashboard.test.jsx
```

Integration Tests

- API endpoint flows (create prompt → test → view results)
- Authentication flows (register → login → access protected routes)
- Version creation on prompt edit
- LLM API integration (mocked responses)
- Tenant isolation verification

End-to-End Tests (Optional for MVP)

- Playwright or Cypress for critical user flows
- Test: Register → Create Prompt → Test Prompt → View Results
- Run in CI before deployment

5.2.2 Quality Checklist

Code Quality:

- [ ] All Python code passes Black + Flake8
- [ ] All JavaScript code passes ESLint + Prettier
- [ ] No console.log statements in production code
- [ ] All API endpoints have tests
- [ ] All UI components have prop types/TypeScript

Security:

- [ ] No hardcoded secrets in code
- [ ] All database queries filter by tenant_id
- [ ] CSRF protection enabled
- [ ] HTTPS enforced in production
- [ ] API rate limiting implemented
- [ ] Input validation on all endpoints

Performance:

- [ ] Database queries optimized (N+1 queries eliminated)

- [ ] Proper database indexes on frequently queried fields
- [ ] Frontend code splitting implemented
- [ ] Images optimized and lazy-loaded
- [ ] API responses cached where appropriate
- [ ] Lighthouse score >90 for key pages

Accessibility:

- [ ] All interactive elements keyboard accessible
- [ ] Semantic HTML used throughout
- [ ] ARIA labels on icon buttons
- [ ] Color contrast meets WCAG AA
- [ ] Screen reader tested

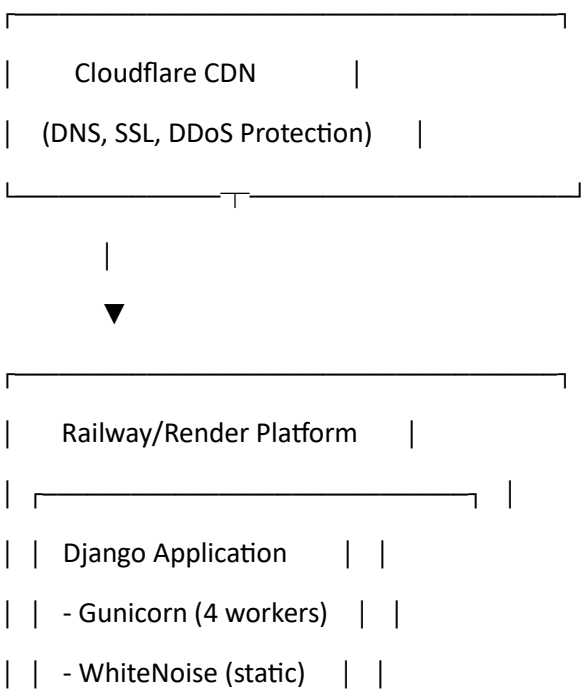
Browser Compatibility:

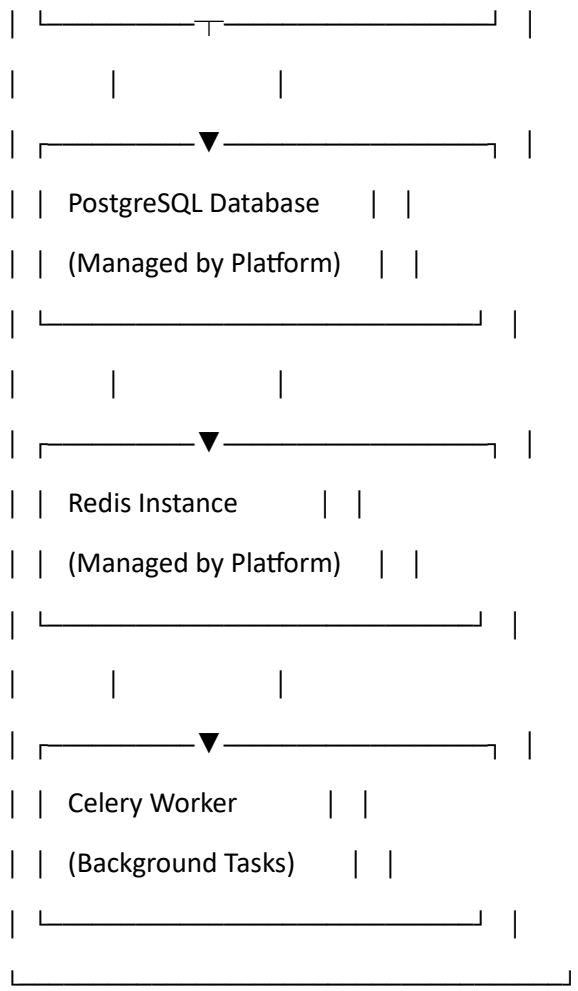
- [ ] Chrome (latest 2 versions)
- [ ] Firefox (latest 2 versions)
- [ ] Safari (latest 2 versions)
- [ ] Edge (latest 2 versions)

5.3 Deployment and Rollback Plan

5.3.1 Deployment Architecture

Production Environment:





5.3.2 Deployment Process

Automated Deployment (GitHub Actions):

```
# .github/workflows/deploy.yml
name: Deploy to Production

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
```

```
    - name: Run Backend Tests
      run: |
        cd backend
        pip install -r requirements.txt
        pytest --cov
    - name: Run Frontend Tests
      run: |
        cd frontend
        npm install
        npm test

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Railway
        run: |
          railway up --service backend
          railway up --service worker
```

Manual Deployment Steps:

1. **Pre-Deployment Checklist:**
    - [ ] All tests passing locally
    - [ ] Database migrations created
    - [ ] Environment variables updated
    - [ ] Backup created
    - [ ] Changelog updated
2. **Deployment Commands:**

```
# 1. Build frontend
cd frontend
npm run build
```

```
# 2. Collect static files
cd backend
python manage.py collectstatic --noinput

# 3. Run migrations
python manage.py migrate

# 4. Deploy (Railway CLI)
railway up

# 5. Restart workers
railway restart --service celery-worker
```

3. **Post-Deployment Verification:**
    - [ ] Health check endpoint returns 200
    - [ ] Login flow works
    - [ ] Can create and test a prompt
    - [ ] Analytics dashboard loads
    - [ ] Error monitoring shows no new errors
4. **Smoke Tests:**

```
# Automated smoke test script
curl https://api.promptops.io/health
curl -X POST https://api.promptops.io/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","password":"password"}'
```

5.3.3 Rollback Plan

Quick Rollback (< 5 minutes):

1. **Railway Platform:**
    - Navigate to Deployments tab
    - Click "Rollback" on previous stable deployment
    - System automatically redeploys previous version
2. **Manual Rollback:**

```
# Revert to previous Git commit
git revert HEAD
git push origin main

# Or checkout previous release tag
git checkout v1.2.0
railway up
```

3. **Database Rollback:**

```
# If migration caused issue
python manage.py migrate prompts 0012  # Rollback to migration 0012

# Restore from backup if needed
pg_restore -d promptops_db backup_file.dump
```

Rollback Decision Criteria:

- Critical bugs affecting >50% of users
- Data corruption detected
- Security vulnerability discovered
- Performance degradation >50%
- Complete service outage >5 minutes

Post-Rollback:

1. Investigate root cause
2. Fix issue in development
3. Write regression test
4. Re-deploy with fix

5.3.4 Monitoring & Alerting

Health Checks:

```
# backend/health/views.py
from rest_framework.views import APIView
from rest_framework.response import Response
from django.db import connection
```

```
class HealthCheckView(APIView):
    permission_classes = []

    def get(self, request):
        # Check database
        try:
            connection.ensure_connection()
            db_status = "healthy"
        except Exception:
            db_status = "unhealthy"

        # Check Redis
        try:
            cache.set('health_check', 'ok', 10)
            redis_status = "healthy"
        except Exception:
            redis_status = "unhealthy"

        healthy = db_status == "healthy" and redis_status == "healthy"

        return Response({
            "status": "healthy" if healthy else "unhealthy",
            "database": db_status,
            "cache": redis_status,
            "version": "1.0.0"
        }, status=200 if healthy else 503)
```

Metrics to Monitor:

- Request latency (p50, p95, p99)
- Error rate (4xx, 5xx)
- Database connection pool usage
- Celery queue length

- LLM API latency and costs
- User registration rate
- Active users (DAU, MAU)

Alerting Rules:

- Error rate >5% for 5 minutes → Email alert
- API latency p95 >2s for 10 minutes → Slack alert
- Database connections >80% → Immediate alert
- Service down → Page on-call (when available)

---

6. Alternatives Considered

6.1 Architecture Alternatives

Monolith vs. Microservices

- **Chosen:** Django monolith
- **Alternative:** Microservices (FastAPI services for prompts, users, analytics)
- **Reasoning:** Monolith is simpler for solo developer, faster initial development, easier debugging. Microservices add complexity without clear benefits at MVP scale.

Relational vs. NoSQL Database

- **Chosen:** PostgreSQL (relational)
- **Alternative:** MongoDB (document store)
- **Reasoning:** Prompt data is highly relational (users ← prompts ← versions). PostgreSQL's JSON fields provide flexibility where needed. ACID transactions important for version integrity.

REST vs. GraphQL

- **Chosen:** REST API
- **Alternative:** GraphQL with Graphene-Django
- **Reasoning:** REST is simpler to implement and debug. GraphQL benefits (flexible queries, single endpoint) not critical for MVP. Can add GraphQL later if needed.

6.2 Technology Alternatives

Backend Framework

- **Chosen:** Django
- **Alternatives Considered:**
    - **FastAPI:** Faster, async-native, but lacks built-in admin, ORM maturity

    - **Flask:** Lightweight, but too minimal (need to add too many extensions)
- **Reasoning:** Django's batteries-included approach (admin, ORM, auth) accelerates solo development.

Frontend Framework

- **Chosen:** React
- **Alternatives Considered:**
    - **Vue.js:** Simpler learning curve, but smaller ecosystem
    - **Svelte:** Faster, less boilerplate, but smaller community/job market
    - **Next.js (React):** SSR benefits, but overkill for SPA use case
- **Reasoning:** React has largest ecosystem, most learning resources, best job market.

Hosting Platform

- **Chosen:** Railway or Render
- **Alternatives Considered:**
    - **Heroku:** More expensive, less generous free tier
    - **AWS/GCP/Azure:** More control, but requires DevOps expertise, complex setup
    - **DigitalOcean:** Good price, but requires server management
- **Reasoning:** Railway/Render offer simplicity + reasonable pricing for solo project.

State Management (Frontend)

- **Chosen:** TanStack Query (React Query)
- **Alternatives Considered:**
    - **Redux Toolkit:** More boilerplate, overkill for server-state-heavy app
    - **Zustand:** Good for client state, but doesn't handle server state caching
    - **Context API:** Too basic, no caching or optimistic updates
- **Reasoning:** React Query specializes in server state, reduces code, handles caching/refetching automatically.

6.3 Feature Alternatives

Version Control Approach

- **Chosen:** Automatic version on every edit
- **Alternative:** Manual version creation (user clicks "Save as New Version")
- **Reasoning:** Automatic ensures no changes are lost, simpler UX. Can add manual versioning later.

LLM Testing Approach

- **Chosen:** Async (Celery task) with polling
- **Alternative:** Synchronous API call with long timeout
- **Reasoning:** LLM calls can take 5-30 seconds; async keeps API responsive, prevents timeouts.

Multi-Tenancy Implementation

- **Chosen:** Single database with tenant_id column
- **Alternative:** Separate database per tenant
- **Reasoning:** Single DB is simpler, cheaper at small scale. Can shard later if needed.

---

7. Open Questions and Risks

7.1 Open Questions

Technical Questions:

1. **LLM API Cost Management:**
    - How to prevent users from running up massive bills?
    - **Mitigation:** Implement per-tenant monthly budgets, show cost estimates before running tests, alert at 80% of budget.
2. **Prompt Content Size Limits:**
    - What's the maximum prompt size to support?
    - **Mitigation:** Start with 100KB limit (far exceeds typical prompts), can increase based on feedback.
3. **Real-time Collaboration:**
    - Should we add WebSockets for live updates (comments, test results)?
    - **Decision:** Not in MVP; use polling initially, evaluate WebSockets in Phase 2.
4. **Version Storage Optimization:**
    - Should we store full content or diffs for versions?
    - **Decision:** Store full content initially (simpler), optimize to diffs if storage becomes issue.
5. **GitHub Sync Direction:**
    - Should sync be bidirectional or one-way (export-only)?
    - **Decision:** One-way export initially, add import based on user feedback.

Product Questions:

1. **Pricing Model:**
    - How to price the SaaS? Per-user, per-prompt, per-test-run?

    - **Research Needed:** Survey potential users, analyze competitor pricing.
2. **Free Tier Limits:**
    - What limits for free tier? (50 prompts, 1000 tests/month reasonable?)
    - **Decision:** Start generous, adjust based on usage patterns.
3. **Prompt Privacy:**
    - Should users be able to share prompts publicly or across tenants?
    - **Decision:** Not in MVP; all prompts private to tenant initially.
4. **AI-Powered Features:**
    - Should we add prompt optimization suggestions, auto-testing?
    - **Decision:** Not in MVP; focus on core management first.

7.2 Risks and Mitigation

Risk 1: LLM API Rate Limits

- **Impact:** High - Users can't test prompts if APIs are rate-limited
- **Probability:** Medium
- **Mitigation:**
    - Implement exponential backoff with jitter
    - Queue requests during high usage
    - Cache common test results
    - Support multiple LLM providers for redundancy

Risk 2: Solo Developer Bandwidth

- **Impact:** High - Features take longer than planned
- **Probability:** High
- **Mitigation:**
    - Ruthlessly prioritize MVP features
    - Use proven libraries instead of building from scratch
    - Accept technical debt in non-critical areas
    - Plan 20% buffer in timeline estimates

Risk 3: Database Performance at Scale

- **Impact:** Medium - Slow queries as data grows
- **Probability:** Medium (if successful)
- **Mitigation:**

    - Proper indexing from start
    - Regular query performance monitoring
    - Implement caching strategically
    - Plan for read replicas if needed

Risk 4: Security Vulnerabilities

- **Impact:** Critical - Data breach would be catastrophic
- **Probability:** Low (with good practices)
- **Mitigation:**
    - Follow OWASP guidelines
    - Regular dependency updates
    - Security code review before launch
    - Bug bounty program post-launch

Risk 5: LLM API Cost Overruns

- **Impact:** High - Could make product unsustainable
- **Probability:** Medium
- **Mitigation:**
    - Strict per-tenant budgets
    - Cost monitoring dashboard
    - Pass costs to users (pricing model)
    - Implement cost alerts

Risk 6: User Adoption

- **Impact:** High - Product fails without users
- **Probability:** Medium (any new product)
- **Mitigation:**
    - Beta test with target users early
    - Gather feedback and iterate quickly
    - Focus on solving real pain points
    - Marketing and community building

Risk 7: Competitor Emergence

- **Impact:** Medium - Established players could add similar features
- **Probability:** High (hot space)

- **Mitigation:**
    - Move fast to capture early adopters
    - Focus on specific niche initially
    - Build community and brand
    - Superior UX as differentiator

Risk 8: Technical Debt Accumulation

- **Impact:** Medium - Slower feature development over time
- **Probability:** High (inevitable in rapid development)
- **Mitigation:**
    - Regular refactoring sprints (every 4 weeks)
    - Code review checklist
    - Automated testing to prevent regressions
    - Documentation as you go

7.3 Assumptions

This design assumes:

1. **User Base:** Primarily English-speaking users with technical background
2. **Scale:** <1000 active users in first 6 months
3. **Budget:** $50-100/month for hosting and services
4. **Time:** Solo developer can dedicate 15-20 hours/week consistently
5. **LLM APIs:** OpenAI and Anthropic APIs remain available and stable
6. **Market:** Demand exists for prompt management tooling
7. **Compliance:** Basic security measures sufficient (not healthcare/finance industry)
8. **Browser Support:** Modern browsers only (no IE11)

7.4 Success Metrics

Technical Metrics:

- API uptime: >99%
- P95 latency: <200ms
- Test coverage: >70%
- Zero critical security vulnerabilities

Product Metrics (6 months post-launch):

- 100+ active organizations

- 500+ registered users
- 10,000+ prompts created
- 100,000+ test runs executed
- 20% monthly active user retention
- Average 3+ prompts per user

Business Metrics:

- 10+ paid customers (if monetizing)
- $500+ MRR (Monthly Recurring Revenue)
- <$2 customer acquisition cost
- Net Promoter Score (NPS) >40

Appendix

A. Glossary of Terms

- **Tenant:** An organization account in the multi-tenant system
- **Version:** A snapshot of a prompt's content at a specific time
- **Test Run:** Execution of a prompt against an LLM with recorded results
- **Pipeline:** Automated workflow for testing and deploying prompts
- **LLM Provider:** Service offering large language models (OpenAI, Anthropic)
- **Token:** Unit of text processing for LLMs (roughly 4 characters)
- **Sandbox:** Interactive environment for testing prompts
- **Diff:** Comparison showing changes between two versions

B. API Endpoint Quick Reference

Authentication:

POST /api/auth/register - Register new user + tenant

POST /api/auth/login - Login and get tokens

POST /api/auth/refresh - Refresh access token

POST /api/auth/logout - Logout (blacklist token)

Prompts:

GET /api/prompts - List prompts

POST /api/prompts - Create prompt

GET /api/prompts/:id - Get prompt details

PATCH /api/prompts/:id - Update prompt (creates version)

DELETE /api/prompts/:id - Archive prompt

Versions:

GET /api/prompts/:id/versions - List versions

GET /api/prompts/:id/versions/:version_id - Get version

POST /api/prompts/:id/revert - Revert to old version

Testing:

POST /api/prompts/:id/test - Run test (async)

GET /api/tasks/:task_id - Check task status

GET /api/prompts/:id/test-history - Test run history

Analytics:

GET /api/analytics/summary - Overall stats

GET /api/analytics/prompts/:id - Prompt-specific stats

Teams:

GET /api/teams - List teams

POST /api/teams - Create team

GET /api/teams/:id - Get team

PATCH /api/teams/:id - Update team

DELETE /api/teams/:id - Delete team

POST /api/teams/:id/members - Add member

DELETE /api/teams/:id/members/:user_id - Remove member

Users:

GET /api/users - List users (in tenant)

GET /api/users/:id - Get user

PATCH /api/users/:id - Update user

DELETE /api/users/:id - Deactivate user

C. Database Migration Commands

```
# Create new migration
python manage.py makemigrations

# View migration SQL
python manage.py sqlmigrate app_name migration_number

# Apply migrations
python manage.py migrate

# Rollback migration
python manage.py migrate app_name previous_migration_number

# Show migration status
python manage.py showmigrations
```

D. Useful Development Commands

```
# Backend
python manage.py runserver            # Start dev server
python manage.py shell                # Django shell
python manage.py createsuperuser      # Create admin user
python manage.py test                 # Run tests
pytest --cov                          # Run tests with coverage
black .                               # Format code
flake8                                # Lint code

# Frontend
npm run dev                           # Start dev server
npm run build                         # Production build
npm test                              # Run tests
npm run lint                          # Lint code
```

```
npm run format                  # Format with Prettier

# Database
docker-compose up -d            # Start PostgreSQL + Redis
docker-compose down             # Stop services
docker-compose logs -f db       # View database logs

# Celery
celery -A config worker -l info     # Start worker
celery -A config beat -l info       # Start scheduler
flower -A config                    # Celery monitoring UI
```

Document Revision History

Version	Date	Author	Changes
1.0	2024-03-15	Initial	Complete design document for MVP

End of Design Document

This comprehensive design document provides everything needed to implement PromptOps as a solo developer project. Refer back to specific sections as you progress through implementation phases. Good luck! 🚀